

ECHONET Lite Tutorial

神奈川工科大学 スマートハウス研究センター

2019.11.11

履歴

Date	Description
2019.11.11	Alpha version release

目次

- [Abstract](#)
- [Prerequisite](#)
- [1. 基本編](#)
 - [1.1 基本コンセプト](#)
 - [1.2 メッセージのフレームフォーマット](#)
 - [1.3 基本シーケンス](#)
 - [1.4 Node Profileと機器探索](#)
- [2. 実用編](#)
 - [2.1 規格書](#)
 - [2.2 ESV](#)
 - [2.3 EOJ](#)
 - [2.4 EPC](#)
 - [2.5 EDT](#)
 - [2.6 機器構成](#)
 - [2.7 起動時処理](#)
 - [2.8 異常処理](#)
 - [2.9 プロパティの値と実装](#)
 - [2.10 複数命令の処理](#)
 - [2.11 スマート電力量メータ](#)
- [3. 発展編](#)
 - [3.1 SetGet](#)
 - [3.2 現実的な実装について](#)
 - [3.3 遠隔操作設定プロパティ](#)
 - [3.4 送信専用ノード](#)
 - [3.5 ECHONET Liteのバージョン間の違い](#)
- [4. 認証制度](#)
 - [4.1 エコネットコンソーシアム](#)
 - [4.2 ECHONET Lite認証](#)
 - [4.3 ECHONET Lite AIF認証](#)
- [付録. メーカーコード](#)

Abstract

このドキュメントはECHONET Liteプロトコルの技術的なTutorialである。基本編を理解すれば、簡単な制御を実行できる。実用編を理解すれば、ECHONET Liteはほぼわかったことになる。発展編は製品レベルを想定した内容である。第4章ではAIFを含む認証制度について簡単に説明する。

Prerequisite

IP NetworkとUDP通信の基本知識が必要である。

Terminology

Term	Description
HEMS	Home Energy Management System
メッセージ	一定の形式に従って記述された、機器間で送受信されるひとまとまりのデータ。意味によってリクエスト、レスポンス、通知と呼ぶことがある。
機器	ECHONET Liteプロトコルを実装した機器
EHD	ECHONET Lite Header
TID	Transaction Id
SEOJ	Source ECHONET Lite Object
DEOJ	Destination ECHONET Lite Object
ESV	ECHONET Lite Service
OPC	Operation Count
EPC	ECHONET Lite Property Code
PDC	Property Data Count
EDT	ECHONET Lite Data

1. 基本編

1.1 基本コンセプト

- ECHONET LiteはHEMSを構成するコントローラや機器間の通信プロトコルである
- ECHONET Liteは通信の制御役（coordinator）が不要であり、システム構成を管理する機器も不要である
- システムは通常1台のコントローラと複数の機器で構成されるが、プロトコル的にはコントローラと機器は対等である
- ECHONET LiteはUDP レイヤー上のプロトコル
 - 受信ポート番号: 3610
 - マルチキャストアドレス: 224.0.23.0 (IPv4), ff02::1 (IPv6)
 - メッセージは全てバイナリデータで定義されている
- 基本操作はプロパティの取得 設定 通知 である
 - 機器の状態や機能はプロパティとして定義されている

- プロパティの値を取得するにはGet命令を使う
- プロパティの値を設定する（操作する）にはSet命令を使う
- プロパティの値を通知するINF命令を使う

1.2 メッセージのフレームフォーマット

ECHONET Liteのメッセージはバイナリーデータである。メッセージのフレームフォーマットを以下に示す。

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| EHD | TID | SEOJ | DEOJ | ESV | OPC | EPC1 | PDC1 | EDT1 | . . . | EPCn | PDCn | EDTn |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

Element	Data size (byte)	Description
EHD	2	ECHONET Lite Header (0x1081)
TID	2	Transaction Id
SEOJ	3	Source ECHONET Lite Object
DEOJ	3	Destination ECHONET Lite Object
ESV	1	ECHONET Lite Service
OPC	1	Operation Count
EPC	1	ECHONET Lite Property Code
PDC	1	Property Data Count
EDT	N	ECHONET Lite Data

基本編では話を簡単にするために OPC=1 とする。この場合のフレームフォーマットを以下に示す。

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| EHD | TID | SEOJ | DEOJ | ESV | OPC=0x01 | EPC | PDC | EDT |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

EHD: ECHONET Lite Header

ECHONET Liteのメッセージであることを示す固定値（0x1081）である。

TID: Transaction ID

メッセージのリクエストとレスポンスの対応をつけるためのデータ。リクエストの送信者はメッセージのTIDに任意の値（通常はリクエストの送信ごとにインクリメントした値）を設定してメッセージを送信する。リクエストの受信者は受信したメッセージのTIDの値をレスポンスのメッセージのTIDに設定してメッセージを返信する。

EOJ: ECHONET Lite Object

EOJは送信元または送信先の機器を示す3byteのデータである。送信元はSEOJ(Source EOJ)、送信先はDEOJ(Destination EOJ)と呼ぶ。EOJの上位2byteで機器の種類を示す。EOJの3byte目は同一種類の機器が複数存在する場合に機器を特定するためのコードである。詳細は2.3 EOJで説明する。

EOJの上位2byteで表す機器の種類の例を以下に示す。

EOJ (上位2byte)	機器名
0x0130	家庭用エアコン
0x027D	蓄電池
0x0288	低圧スマート電力量メータ
0x0290	一般照明
0x05FF	コントローラ
0x0EF0	ノードプロファイル

ESV: ECHONET Lite Service

ESV はメッセージのサービスの種類を示す。詳細は2.2 ESVで説明する。以下にESVの例を示す。

ESV	Symbol	Type	Description
0x62	Get	request	Get property value
0x72	Get_Res	response	Response property value
0x61	SetC	request	Set property value
0x71	Set_Res	response	Ack to SetC command(*)
0x73	INF	notification	Inform property value

(*) Set_Res はSetCを受信したことに対するレスポンスであって、命令を実行したことに対するレスポンスではない。命令が実行されたかどうかを確認するにはGetコマンドを実行する。

OPC: Operation Count

1つの命令(operation)は、後述するEPC, PDC, EDTのセットで構成される。1メッセージで複数の命令を送信することが可能で、OPCは1メッセージ内の命令の個数を示す。

EPC: ECHONET Lite Property Code

機器の設定・状態・機能はプロパティと呼ばれ、1バイトのデータで定義されている。

PDC: Property Data Count

後述するEDTのデータサイズを示す。ESV=0x62(Get)の場合はEDTが不要なのでPDC=0となる。

EDT: ECHONET Lite Data

プロパティの値である。データサイズは可変長。

家庭用エアコン(EOJ=0x0130)のEPC, EDTの例

EPC	プロパティ名	PDC	EDT
0x80	動作状態	1	0x30:ON, 0x31:OFF
0xA0	風量設定	1	8段階のレベル。0x31...0x38
0xB0	運転モード設定	1	0x42:冷房, 0x43:暖房
0xB3	温度設定値	1	0~50
0xBB	室内温度計測値	1	-127~125

照明(EOJ=0x0290)のEPC, PDC, EDTの例

EPC	プロパティ名	PDC	EDT
0x80	動作状態	1	0x30:ON, 0x31:OFF
0xB0	照度レベル設定	1	0 ~ 100%
0xB6	点灯モード設定	1	0x42:通常灯, 0x43:常夜灯, 0x45:カラー灯
0xC0	RGB設定	3	RGB各1byte, 0 ~ 255

1.3 基本シーケンス

家庭用エアコンの動作状態(EPC=0x80)を例にして基本シーケンスの説明をする。

プロパティ値の取得

プロパティの取得はリクエストに ESV=0x62(Get) を、レスポンスに ESV=0x72(Get_Res) を利用する。以下の例ではエアコンがOFF(0x31)であることがわかる。

```

リクエスト：コントローラ(0x05FF01) -> エアコン(0x013001)
1081 0001 05FF01 013001 62 01 80 00
<---> <---> <-----> <-----> <-> <-> <-> <->
EHD  TID  SE0J  DE0J  ESV OPC EPC PDC

レスポンス：エアコン(0x013001) -> コントローラ(0x05FF01)
1081 0001 013001 05FF01 72 01 80 01 31
<---> <---> <-----> <-----> <-> <-> <-> <-> <->
EHD  TID  SE0J  DE0J  ESV OPC EPC PDC EDT

```

プロパティ値の設定

プロパティの設定はリクエストに ESV=0x61(SetC) を、レスポンスに ESV=0x71(SetC_Res) を利用する。SetC_ResはSetCコマンドを受信したことに対するACKの意味であり、実際にプロパティが設定されたかどうかは別途確認が必要である。以下はエアコンの動作状態をON(EDT=0x30)にする例である。

```

リクエスト：コントローラ(0x05FF01) -> エアコン(0x013001)
1081 0002 05FF01 013001 61 01 80 01 30
<---> <---> <-----> <-----> <-> <-> <-> <->
EHD  TID  SE0J  DE0J  ESV OPC  EPC  PDC  EDT

```

```

レスポンス：エアコン(0x013001) -> コントローラ(0x05FF01)
1081 0002 013001 05FF01 71 01 80 00
<---> <---> <-----> <-----> <-> <-> <-> <->
EHD  TID  SE0J  DE0J  ESV OPC  EPC  PDC

```

プロパティの通知

プロパティの通知は ESV=0x73(INF)を利用する。以下はエアコンの動作状態(EPC=0x80)がON(EDT=0x30)になったことを通知する例である。

```

通知：エアコン(0x013001) -> コントローラ(0x05FF01)
1081 0001 013000 05FF01 73 01 80 01 30
<---> <---> <-----> <-----> <-> <-> <-> <->
EHD  TID  SE0J  DE0J  ESV OPC  EPC  PDC  EDT

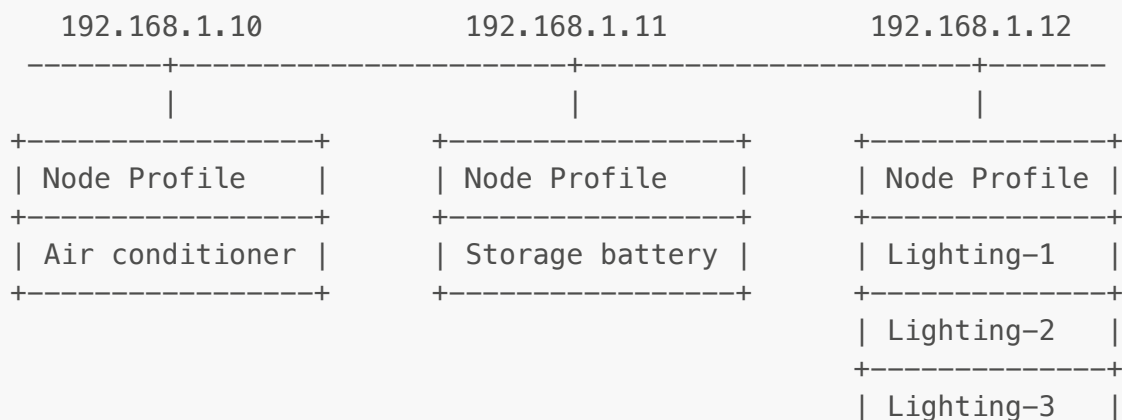
```

1.4 Node Profileオブジェクトと機器探索

ECHONET Liteで通信を行うには、通信相手のIPアドレスとEOJを知る必要がある。ここではネットワーク内のどのIPアドレスにどのような機器オブジェクト(EOJ)が存在するかを調べる方法を説明する。

ECHONET Lite機器は、IP通信を行うNode Profileオブジェクトと機器固有の機能を実現する機器オブジェクトで構成される。Node ProfileオブジェクトはIPアドレスに対応して1つ存在する。機器オブジェクトの具体例としては家庭用エアコン、蓄電池、一般照明などがある。一つのIPアドレスに機器オブジェクトが複数個存在する構成も可能である。

Node Profileオブジェクトと機器オブジェクトの構成の例を以下に示す。IP address 192.168.1.10には家庭用エアコンが1台、IP address 192.168.1.11には蓄電池が1台、IP address 192.168.1.12には一般照明が3台存在する。



+-----+

ECHONET Lite通信の送信元(SEOJ)や送信先(DEOJ)は上記のNode Profileオブジェクトや機器オブジェクトを指定する。EOJは3byteで構成され、上位2byteはオブジェクトの種類（例：0x0EF0:Node Profile,0x0130:家庭用エアコン, 0x027D:蓄電池, 0x0290:一般照明)を示し、3 byte目は同一種類のオブジェクトを区別するための番号である。Node ProfileオブジェクトのEOJは常に0x0EF001である。以下にEOJの例を示す。

192.168.1.10	192.168.1.11	192.168.1.12
-----+ +-----+	-----+ +-----+	-----+ +-----+
Node Profile	Node Profile	Node Profile
EOJ:0x0EF001	EOJ:0x0EF001	EOJ:0x0EF001
+-----+	+-----+	+-----+
Air conditioner	Storage battery	Lighting-1
EOJ:0x013001	EOJ:0x027D01	EOJ:0x029001
+-----+	+-----+	+-----+
		Lighting-2
		EOJ:0x029002
		+-----+
		Lighting-3
		EOJ:0x029003
		+-----+

Node profileオブジェクトはそのIPアドレスに存在する機器オブジェクトの情報をプロパティ「自ノードインスタンスリストS」(EPC=0xD6)として持つ。このプロパティのデータフォーマットは以下のとおり。

1byte目: 機器オブジェクトの個数

2byte目以降: EOJの列挙

例えば、上記のIPアドレス(192.168.1.10, 192.168.1.11, 192.168.1.12)に対応するそれぞれのNode Profileオブジェクトの自ノードインスタンスリストSの値は、「01013001」「01027D01」「03028801028802028803」となる。したがってネットワーク内の機器に「DEOJ=0x0EF001, ESV=0x62(Get), EPC=0xD6」のメッセージを送れば、それぞれのIPアドレスにどのような機器が存在するかがわかる。

ところでECHONET Lite機器のIPアドレスは通常DHCPで設定されるので、通信相手のIPアドレスがわからない。そこでUDPのマルチキャストを利用してネットワーク内の全ての機器にメッセージを送信する。

ECHONET LiteはUDPマルチキャストアドレスとして224.0.23.0を登録してある。

つまり、**「DEOJ=0x0EF001, ESV=0x62(Get), EPC=0xD6」のメッセージを IP=224.0.23.0 にマルチキャスト**することで機器探索を行うことができる。

以下に機器探索のリクエストとレスポンスの例を示す。

```

リクエスト (Destination IP: 224.0.23.0)
1081 0001 05FF01 0EF001 62 01 D6 00
<--> <--> <----> <----> <-> <-> <-> <->
EHD TID SE0J DE0J ESV OPC EPC PDC

レスポンス (Source IP: 192.168.1.10)
1081 0001 0EF001 05FF01 72 01 D6 04 01013001
レスポンス (Source IP: 192.168.1.11)
1081 0001 0EF001 05FF01 72 01 D6 04 01027D01
レスポンス (Source IP: 192.168.1.12)
1081 0001 0EF001 05FF01 72 01 D6 0A 03029001029002029003
<--> <--> <----> <----> <-> <-> <-> <-> <----->
EHD TID SE0J DE0J ESV OPC EPC PDC EDT

```

2. 実用編

2.1 規格書

ECHONET Liteの規格書は以下の3つのドキュメントで構成され、[エコネットコンソーシアムのホームページ](#)からダウンロードできる。

- ECHONET Lite規格書
- ECHONET Liteシステム設計指針
- APPENDIX ECHONET機器オブジェクト詳細規定

ECHONET Lite規格書

「ECHONET Lite規格書」は規格の本体である。第1部から第4部で構成され、約300ページの分量がある。本チュートリアルを理解すれば目を通す機会は少ない。必要に応じて第2部を参照すること。

ECHONET Lite規格の最新版は2019年8月時点でVer. 1.13である。機器が実装しているECHONET LiteのバージョンはNode profileオブジェクト(EOJ=0x0EF001)のVersion情報プロパティ(EPC=0x82)として取得できる。本TutorialはV1.13を前提に記述している。通常の機器制御においてECHONET LiteのVersionを意識する必要はない。

ECHONET Liteシステム設計指針

「ECHONET Liteシステム設計指針」は実装のガイドラインである。本チュートリアルを理解すれば目を通す機会は少ない。

APPENDIX ECHONET機器オブジェクト詳細規定

「APPENDIX ECHONET機器オブジェクト詳細規定」(以下APPENDIXと略す)には機器オブジェクト毎のプロパティの仕様が記述されている。機器を制御するためにはこの規格書を参照する必要がある。半年に1度程度改定があり、そのバージョンをReleaseと表記している。Releaseは"A"から始まり2019年8月時点では"L"が最新である。プロパティの仕様はbackward compatibilityが考慮されていない(*)ため、コントローラは機器が実装しているReleaseの仕様に合わせてメッセージを送信したり、受信したメッセージを解釈する必

要がある。機器が実装しているReleaseのバージョンは機器オブジェクトの規格Version情報プロパティ (EPC=0x82)として取得できる。

(*) 例：電動ブラインド(EOJ=0x0260)の開度レベル(EPC=0xE1)はRelease A~Cでは8段階のレベル (0x31~0x38)であったが、Release D以降では数値(0~100%)になった。

なお、ノードプロファイルオブジェクトのプロパティ情報は「ECHONET Lite規格書」の第2部6.10に記載されている。

以下にAPPENDIX ECHONET機器オブジェクト詳細規定の中の一般照明の一部を示す。プロパティの情報が表にまとめられている。この表の後に各プロパティの説明がある。ここに表には記載されていない追加情報 (例：未設定に対応する値は0xFF) が記述されている場合があるので注意すること。

3. 3. 30 一般照明クラス規定

クラスグループコード : 0x02
クラスコード : 0x90
インスタンスコード : 0x01~0x7F (0x00 : 全インスタンス指定コード)

プロパティ名称	EPC	プロパティ内容	データ型	データサイズ	単位	アクセスルール	必須	状態時 アナウンス	備考
		値域(10進表記)							
動作状態	0x80	ON/OFFの状態を示す。	unsigned char	1 Byte	-	Set	○	○	
		ON=0x30, OFF=0x31				Get	○		
照度レベル設定	0xB0	照度レベルを%で示す。	unsigned char	1 Byte	%	Set/Get			
		0x00~0x64(0~100%)							

2.2 ESV (ECHONET Lite service code)

ECHONET Liteのメッセージの種類をサービスと呼び、ESV(ECHONET Lite service code)として1byteのデータで定義している(reference:第2部3.2.5, 4.2.3)。サービスは以下の4種類に分類される。サービスの実装については第3章を参照のこと。

- リクエスト: プロパティ値のREAD, WRITE, 通知のリクエスト
 - ESV=0x60, 0x61, 0x62, 0x63, 0x6E
- レスポンス (正常): リクエストを正常に受け付けた場合のレスポンス (ESV=0x71, 0x72, 0x73, 0x7E)
- レスポンス (異常): リクエストを正常に受け付けられない場合のレスポンス。不可応答:SNA(*)と呼ぶ。(ESV=0x50, 0x51, 0x52, 0x53, 0x5E)
- 自発的通知: プロパティ値の自発的通知 (ESV=0x73, 0x74)

(*) 規格書には明記されていないが、Service Not Availableの略だと推定する。

リクエスト&レスポンス

ESV リクエスト	Symbol	Description	ESV レスポンス (正常)	Symbol	ESV レスポンス (異常)	Symbol	Note
0x60	SetI(*1)	Write(応答不要)	-	-	0x50	SetI_SNA	お勧めしない

ESV リクエスト	Symbol	Description	ESV レスポンス (正常)	Symbol	ESV レスポンス (異常)	Symbol	Note
0x61	SetC(*2)	Write(応答必要)	0x71	Set_Res(*3)	0x51	SetC_SNA	よく使われる
0x62	Get	Read	0x72	Get_Res	0x52	Get_SNA	よく使われる
0x63	INF_REQ(*4)	通知	0x73	INF(*5)	0x53	INF_SNA(*6)	お勧めしない
0x6E	SetGet(*7)	Write&Read	0x7E	SetGet_Res	0x5E	SetGet_SNA	お勧めしない

(*1) 規格書には明記されていないが、Set with Ignorance の略だと推定する。

(*2) 規格書には明記されていないが、Set with Confirmation の略だと推定する。

(*3) Set_ResはSetCコマンドを受信したことに対する確認(ACK)の意味である。

(*4) INF_REQをマルチキャスト送信することはネットワークの輻輳の可能性があるため望ましくない(reference: システム設計指針2.4.1)。

(*5) INF_REQのレスポンスとしてのINFはマルチキャスト(reference:第2部4.2.3.5)。

(*6) INF_SNAはユニキャスト(reference:第2部4.2.3.5)。

(*7) SetGetのフレーム構成は1.2で説明したものと異なる。3.1 SetGetで説明する。

- Setに関してはSetCを使えばよく、SetIを積極的に使う理由はない。
- INF_REQはレガシーコマンドである。使う必要がない。
- SetGetのコマンドの設計思想(reference:第2部4.2.3.4)は複数のSetやGetの命令を1メッセージで送信することであり、それぞれの命令の実行順は機器依存としているため、Setしたプロパティの値を確認するためには利用できない。またエラー処理が複雑である。したがって無理にSetGetを使うよりもSetとGetを別々に実行することをお勧めする。

自発的通知

オブジェクト（Node profileオブジェクトまたは機器オブジェクト）が自発的にINFまたはINFCを利用してプロパティ値を伝える場合について説明する(reference:第2部6.2.4)。

ESV 通知	Symbol	Description	ESV レスポンス	Symbol	Note
0x73	INF	通知(応答不要)	-	-	よく使われる
0x74	INFC	通知(応答必要)	INFC_Res	0x7A	あまり使われない ユニキャストのみ

- 「状態時アナウンス」が必須であるプロパティを実装する場合、プロパティ値が変わった場合には必ずマルチキャストでINFを送信する。DEOJ にはノードプロファイル(0x0EF001)を設定する。
- INFCはユニキャストのみ(reference:第2部4.2.3.6)。

ESVとアクセスルール

APPENDIXのアクセスルール欄と必須欄の説明を以下に記述する。

- アクセスルール欄はどのサービス(ESV)に対応しているかを示す。(reference:第2部6.2.5)
 - Set: 0x60(SetI), 0x61(SetC), 0x6E(SetGet)
 - Get: 0x62(Get), 0x6E(SetGet), 0x63(INF_REQ)
 - Anno: 0x63(INF_REQ)
 - アクセスルールがAnnoであるプロパティは、Node profileのEPC=0xD5のみである
- 必須欄はアクセスルールのSetやGetに対応するサービスの実装に関する情報である。アクセスルールにSetやGetの記載があり、必須欄が「○」の場合はそのプロパティが必ず実装されていることを示し、空白の場合はそのプロパティが実装されるかどうかは機器依存であることを示す。
- 状態時アナウンス欄に「○」が記載されているプロパティを実装する場合は、プロパティ値に変化があった時にINFまたはINFCをマルチキャスト(DEOJ=0x0EF001)送信する。(reference:第2部6.2.4)

2.3 EOJ (ECHONET object)

ECHONET Liteのメッセージの送信または受信を行う主体をECHONET オブジェクト(EOJ)呼ぶ。EOJは3byteで構成され、1byte目をクラスグループコード、2byte目をクラスコード、3byte目をインスタンスコードと呼ぶ(reference:第2部3.2.4)。クラスグループコードは機器種別の大分類（例 0x01:空調関連クラスグループ）クラスコードは小分類（例 空調機器関連クラスグループの0x30は家庭用エアコン）と定義されているが、実用的にはこの2byteで機器種別を表しているのので、このドキュメントではEOJの上位2byteをDevice typeと呼ぶ。インスタンスコードは同一Device typeの機器が複数存在する場合に機器を特定するためのコードで、機器側が設定する値(1~255)である。

- Device type: 0x0F00~0FFF はユーザーが自由に定義して使えるコードである。
- DEOJのインスタンスコードの0x00はワイルドカードで、同一種類の機器全てを送信先に設定する。例えばDEOJ=0x029000とすると、送信先IPアドレスの全ての照明に対して同一のメッセージを送ることができる。
- Node profileのEOJは0x0EF001（ただし送信専用ノード(3.4 送信専用ノード参照)の場合は0x0EF002）。
- インスタンスコードは機器が設定するが、その値に関しては規則やガイドラインは存在しない。多くの場合その値は0x01であり、同一Device typeの機器が複数存在する場合にはインクリメントすることが多いが、それを前提として制御してはいけない。

以下にDevice typeの例を示す

Device type	Device name	Device type	Device name	Device type	Device name
0x000D	照度センサ	0x0272	瞬間式給湯器	0x0288	低圧スマート電力量メータ
0x0011	温度センサ	0x0273	浴室暖房乾燥機	0x028A	高圧スマート電力量メータ
0x0012	湿度センサ	0x0279	住宅用太陽光発電	0x0290	一般照明
0x0022	電力量センサ	0x027A	冷温水熱源機	0x0291	単機能照明

Device type	Device name	Device type	Device name	Device type	Device name
0x002D	気圧センサ	0x027B	床暖房	0x02A1	電気自動車充電器
0x0130	家庭用エアコン	0x027C	燃料電池	0x02A6	ハイブリッド給湯器
0x0133	換気扇	0x027D	蓄電池	0x03B7	冷凍冷蔵庫
0x0135	空気清浄機	0x027E	電気自動車充放電器	0x03B8	オープンレンジ
0x0260	電動ブラインド	0x0280	電力量メータ	0x03B9	クッキングヒータ
0x0263	電動雨戸	0x0281	水力量メータ	0x03D3	洗濯乾燥機
0x026B	電気温水器	0x0282	ガスメータ	0x05FD	スイッチ
0x026F	電気錠	0x0287	分電盤メータリング	0x05FF	コントローラ

2.4 EPC

2.4.1 機器オブジェクトのEPC

機器オブジェクトのプロパティは1バイトのEPCで定義され、Super classのプロパティ (reference:APPENDIX第2章)と、機器オブジェクトごとのプロパティ (reference:APPENDIX第3章)で定義されている。Super classのプロパティは全ての機器で共通であり、機器オブジェクトごとのプロパティでオーバーライドされる場合がある。

Super classのプロパティを以下に示す。機器オブジェクトごとのプロパティはAPPENDIX第3章を参照すること。

Super class

EPC	プロパティ名称	アクセス ルール	必須	状態時 アナウンス	Note
0x80	動作状態	Set Get	x ○	○	
0x81	設置場所	Set/Get	○	○	(*1)
0x82	規格Version情報	Get	○	x	(*2)
0x83	識別番号	Get	x	x	(*3)
0x84	瞬時消費電力計測値	Get	x	x	
0x85	積算消費電力計測値	Get	x	x	
0x86	メーカー異常コード	Get	x	x	

EPC	プロパティ名称	アクセス ルール	必須	状態時 アナウンス	Note
0x87	電流制限設定	Set/Get	x	x	
0x88	異常発生状態	Get	○	○	
0x89	異常内容	Get	x	x	
0x8A	メーカーコード	Get	○	x	
0x8B	事業場コード	Get	x	x	
0x8C	商品コード	Get	x	x	
0x8D	製造番号	Get	x	x	
0x8E	製造年月日	Get	x	x	
0x8F	節電設定	Set/Get	x	x	
0x93	遠隔操作設定	Set/Get	x	x	(*4)
0x97	現在時刻設定	Set/Get	x	x	
0x98	現在年月日設定	Set/Get	x	x	
0x99	電力制限設定	Set/Get	x	x	
0x9A	積算運転時間	Get	x	x	
0x9D	状態アナウンスプロパティマップ	Get	○	x	
0x9E	Setプロパティマップ	Get	○	x	
0x9F	Getプロパティマップ	Get	○	x	

(*1) プロパティ値は1byte or 17byte。

1byteの場合はbitmap (reference:APPENDIX2.2) 0x00:未設定, 0xFF:不定。

(*2) 実装しているAPPENDIXのRelease version。

1st,2nd byte:0x00(Release Aを除く)。

3rd byte:Release versionをASCII codeで表す。Aのみ小文字(0x61)、B以降は大文字(B:0x42...L:0x4C)。

4th byte:0x00

(*3) 0xFE + メーカーコード + ユニークID(13byte)

(*4) 3.3 遠隔操作設定を参照

EPCは値の範囲で以下のようにカテゴリー分けされている(reference:第2部3.2.7)。

EPC	Category	Example
0x00 ~ 0x7F	reserved	
0x80 ~ 0x9F	Common properties across devices	0x80: 動作状態 0xB0
0xA0 ~ 0xEF	Specific properties to each device	エアコン: 運転モード 照明: 照度レベル

EPC	Category	Example
0xF0 ~ 0xFF	User defined properties	

Super classのプロパティはEPC=0x80~0x9Fのうち以下のEPCを除いたものである。これらは必要に応じて機器オブジェクト側で記述される。

EPC	プロパティ名称
0x90	ONタイマ予約設定
0x91	ONタイマ時刻設定
0x92	ONタイマ相対時間設定
0x94	OFFタイマ予約設定
0x95	OFFタイマ時刻設定
0x96	OFFタイマ相対時間設定
0x9B	(reserved for legacy)
0x9C	(reserved for legacy)

2.4.2 Node profileオブジェクトのEPC

Node profileオブジェクトのプロパティは1バイトのEPCで定義され、Super class(reference:第2部6.10.1)と、Node profileオブジェクトのプロパティ(reference:第2部6.11.1)で定義されている。Node profileオブジェクトのSuper classと機器オブジェクトのSuper classは異なる。Node profileオブジェクトの全て（Super classを含む）のプロパティを以下に示す。

EPC	プロパティ名称	アクセス ルール	必須	状態時 アナウンス	note
0x80	動作状態	Set Get	x ○	○	
0x82	Version情報	Get	○	x	(*1)
0x83	識別番号	Get	○	x	(*2)
0x88	異常発生状態	Get	x	x	
0x89	異常内容	Get	x	x	
0x8A	メーカーコード	Get	○	x	
0x8B	事業場コード	Get	x	x	
0x8C	商品コード	Get	x	x	
0x8D	製造番号	Get	x	x	
0x8E	製造年月日	Get	x	x	

EPC	プロパティ名称	アクセス ルール	必須	状態時 アナウンス	note
0x9D	状態アナウンスプロパティマップ	Get	○	x	
0x9E	Set プロパティマップ	Get	○	x	
0x9F	Get プロパティマップ	Get	○	x	
0xBF	個体識別情報	Set/Get	x	x	
0xD3	自ノードインスタンス数	Get	○	x	(*3)
0xD4	自ノードクラス数	Get	○	x	(*4)
0xD5	インスタンスリスト通知	Anno	○	○	(*5)
0xD6	自ノードインスタンスリストS	Get	○	x	(*5)
0xD7	自ノードクラスリストS	Get	○	x	(*3)

(*1) ECHONET Lite規格のversion。

1バイト目：メジャーバージョン

2バイト目：マイナーバージョン

3バイト目：メッセージ形式をbitmapで示す

4バイト目：0x00

例 0x010D0100: V1.13

(*2) 0xFE + メーカーコード + ユニークID(13byte)。

(*3) ノードプロファイルクラスを含まない。

(*4) ノードプロファイルクラスを含む。

(*5) EPC=0xD5と0xD6のEDTは同じ内容。

Node profileオブジェクトと機器オブジェクトのSuper classの違いを以下に示す。

EPC	Node profile object	機器オブジェクト	Note
0x81	Not applicable	設置場所	
0x82	Version情報	規格Version情報	ECHONET Lite version : Node profile object Appendix version : 機器オブジェクト
0x83	識別番号	識別番号	アクセスルール：必須(Node profile object) アクセスルール：非必須(機器オブジェクト)
0x84	Not applicable	瞬時消費電力計測値	
0x85	Not applicable	積算消費電力計測値	
0x86	Not applicable	メーカー異常コード	
0x87	Not applicable	電流制限設定	
0x88	異常発生状態	異常発生状態	アクセスルール：非必須(Node profile object) アクセスルール：必須(機器オブジェクト)
0x8F	Not applicable	節電動作設定	

EPC	Node profile object	機器オブジェクト	Note
0x93	Not applicable	遠隔操作設定	
0x97	Not applicable	現在時刻設定	
0x98	Not applicable	現在年月日設定	
0x99	Not applicable	電力制限定	
0x9A	Not applicable	積算運転時間	

2.4.3 Atomic operation

いくつかの操作を組み合わせてそれらがひとつの操作に見えるものを情報工学においては atomic operation と呼ぶ。以下に、ECHONET Liteにおいてatomic operationと言えるEPCを示す。2nd EPCを実行(Get)する前に1st EPCの実行(SetC)が必要である。

EOJ	機器 オブジェクト名	1st EPC	プロパティ名	2nd EPC	プロパティ名
0x027E	電気自動車 充放電器	0xCD	車両接続確認	0xC7	車両接続・充放電可否状態
0x0287	分電盤 メータリング	0xB2	積算電力量計測チャンネル 範囲指定（片方向）	0xB3	積算電力量計測値リスト （片方向）
-	-	0xB4	瞬時電流計測チャンネル 範囲指定（片方向）	0xB5	瞬時電流計測値リスト （片方向）
-	-	0xB6	瞬時電力計測チャンネル 範囲指定（片方向）	0xB7	瞬時電力計測値リスト （片方向）
-	-	0xB9	積算電力量計測チャンネル 範囲指定（双方向）	0xBA	積算電力量計測値リスト （双方向）
-	-	0xBB	瞬時電流計測チャンネル 範囲指定（双方向）	0xBC	瞬時電流計測値リスト （双方向）
-	-	0xBD	瞬時電力計測チャンネル 範囲指定（双方向）	0xBE	瞬時電力計測値リスト （双方向）
-	-	0xC5	積算履歴収集日	0xC3	積算電力量計測値履歴 （正方向）
0x0288	低圧スマート 電力量メータ	0xE5	積算履歴収集日 1	0xE2	積算電力量計測値履歴1 （正方向計測値）
-	-	0xE5	積算履歴収集日 1	0xE4	積算電力量計測値履歴1 （逆方向計測値）
-	-	0xED	積算履歴収集日 2	0xEC	積算電力量計測値履歴2 （正方向、逆方向計測値）

EOJ	機器 オブジェクト名	1st EPC	プロパティ名	2nd EPC	プロパティ名
0x028A	高圧スマート 電力量メータ	0xE1	積算履歴収集日	0xC6	需要電力計測値履歴
-	-	0xE1	積算履歴収集日	0xCE	力測積算無効電力量 (遅れ) 計測値履歴
-	-	0xE1	積算履歴収集日	0xE7	積算有効電力量計測値履歴
0x02A1	電気自動車 充電器	0xCD	車両接続確認	0xC7	車両接続・充電可否状態

2.5 EDT

EDTはバイナリーデータであり、その値をどのように解釈するかはEPCごとに異なる。その仕様はAPPENDIXのプロパティ内容、値域、データ型、データサイズ、単位に記述されている。数値、状態、レベルの3種類のデータタイプがよく使われている。

数値

数値の場合、データ型欄にデータのサイズとsigned/unsingedの情報がC言語のキーワードを利用して記述されている。以下にエアコン(EOJ=0x0130)の温度設定値(EPC=0xB3)の記述を例として示す。

温度設定値	0xB3	温度設定値を設定し、設定状態を取得する。 0x00～0x32 (0～50℃)	unsigned char	1 Byte	℃	Set/ Get	○		
-------	------	---	------------------	-----------	---	-------------	---	--	--

少数を含む値を扱う場合は、単位欄に係数が記述されている。例えばエアコンの相対温度設定値(EPC=0xBF)の場合、EDT値が25であれば2.5℃と解釈する。

相対温度設定値	0xBF	エアコン動作中、動作モードにおける目標温度値に対する相対温度設定値を設定し、設定内容を取得する。 0x81～0x7D(-12.7℃～12.5℃)	signed char	1 Byte	0.1 ℃	Set /Get			
---------	------	---	----------------	-----------	----------	-------------	--	--	--

データ型ごとにアンダーフローとオーバーフローに対応する値が定義されている(reference:第2部6.2.2)。例えばGETで取得したEDTのデータ型がunsigned charでその値が0xFF(255)の場合、機器内部の値は254以上であることを示す。

データ型	データサイズ	アンダーフロー	オーバーフロー
signed char	1 byte	0x80	0x7F
signed short	2 byte	0x8000	0x7FFF
signed long	4 byte	0x80000000	0x7FFFFFFF
unsigned char	1 byte	0xFE	0xFF
unsigned short	2 byte	0xFFFE	0xFFFF

データ型	データサイズ	アンダーフロー	オーバーフロー
unsigned long	4 byte	0xFFFFFFFFE	0xFFFFFFFFF

状態

電源のON/OFFやエアコンの運転モードの冷房・暖房などの状態を特定の値に対応させることで、EDTで状態を扱うことができる。以下にエアコン(EOJ=0x0130)の運転モード設定(EPC=0xB0)の記述を例として示す。

運転モード設定	0xB0	自動／冷房／暖房／除湿／送風／その他の運転モードを設定し、設定状態を取得する。 順番に以下のコードが対応。 0x41/0x42/0x43/0x44/0x45/0x40	unsigned char	1 Byte	—	Set/Get	○	○	
---------	------	---	---------------	--------	---	---------	---	---	--

レベル

エアコンの風量設定などの強弱のレベル設定において、レベル値を特定の値に対応させることで、EDTでレベルを扱うことができる。特に8段階のレベルを0x31(Level-1)~0x38(Level-8)にマッピングする定義はよく使われる。以下にエアコン(EOJ=0x0130)の風量設定(EPC=0xA0)の記述を例として示す。

風量設定	0xA0	風量レベルおよび風量自動状態を設定し、設定状態を取得する。 風量レベルは8段階で指定。 風量自動設定=0x41 風量レベル=0x31~0x38	unsigned char	1 Byte	—	Set/Get	○	○	
------	------	--	---------------	--------	---	---------	---	---	--

複数データ

EDTに複数データをパッキングすることもある。以下の照明(EOJ=0x0290)のRGB設定(EPC=0xC0)例では、1バイト目はR、2バイトめはG、3バイト目はBの値であることを示す。。

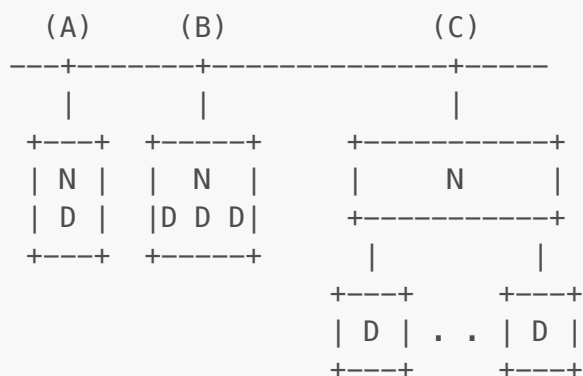
カラー灯モード時 RGB 設定	0xC0	カラー灯モード時のRGB値を設定し、設定状態を取得する。 1Byte 目:R 2Byte 目:G 3Byte 目:B 0x00~0xFF(0~255) 最低輝度=0x00,最高輝度=0xFF	unsigned char×3	3 Byte	—	Set/Get			
-----------------	------	--	-----------------	--------	---	---------	--	--	--

以下の低圧スマート電力量メータ(EOJ=0x0288)の積算電力量計測値履歴1(EPC=0xE2)の例では、最初の2バイトは日にちを示し、それ以降は48個の電力量(4バイト)を示す。

積算電力量計測値履歴1(正方向計測値)	0xE2	積算履歴収集日1と該当収集日の24時間48コマ分(0時0分~23時30分)の正方向の定時積算電力量計測値の履歴データを時系列順に上位バイトからプロパティ値として示す。 1~2バイト目:積算履歴収集日 0x0000~0x0063(0~99) 3バイト目以降:積算電力量計測値 0x00000000~0x05F5E0FF(0~99,999,999)	unsigned short + unsigned long ×48	194 Byte	kWh	Get	○		
---------------------	------	--	------------------------------------	----------	-----	-----	---	--	--

2.6 Node profileオブジェクトと機器オブジェクトの構成

ECHONET Lite機器のNode profileオブジェクトと機器オブジェクト (Device object)の構成は、下図のような3種類が考えられる。



N:Node profile object, D:Device object

(A) 基本構成：製品がNode profileと1つの機器オブジェクトで構成される場合

(B) 一つの製品がNode profileと複数の機器オブジェクトで構成される場合

例：床暖房が、床暖房オブジェクトと冷温水熱源器オブジェクトで構成されている場合

(C) 独立した複数の製品が同一Node profileオブジェクトを共用する場合。Node profileオブジェクトと機器オブジェクト間の接続は特定小電力無線、BluetoothやWiSUN-HANなどのproprietaryな通信も利用可能

例：Panasonic AiSEG, Toshiba Feminity

(B)や(C)の構成の場合、Node profileのEPC=0xD5, D6のInstance listには複数の機器オブジェクトのEOJが記述される。

2.7 起動時処理

2.7.1 機器

機器は起動時やIPアドレス変更時にインスタンスリストをマルチキャスト（宛先IP=224.0.23.0）して、新規の機器がネットワークに参加したことを通知する。

```

Destination IP: 224.0.23.0
SE0J: 0x0EF001
DE0J: 0x0EF001
ESV: 0x73 (INF)
OPC: 1
EPC: 0xD5
EDT: 自ノード内のインスタンスリスト
  
```

例：家庭用エアコンの場合

コントローラ
(IP=192.168.1.1)

エアコン
(IP=192.168.1.2)

(EOJ=0x05FF01)	(EOJ=0x013001)
IP SRC:192.168.1.2, DST:224.0.23.0	
1081 0001 0EF001 0EF001 73 01 D5 04 01013001	
<-----	

2.7.2 コントローラ

コントローラは起動時に機器を探索し、その後操作に必要な情報を各機器から取得する。ここでは最低限必要な動作について説明する。

1. インスタンスリストの取得

ネットワーク内の機器を探索するためにインスタンスリストのリクエストをマルチキャスト（宛先 IP=224.0.23.0）する。インスタンスリストのレスポンスから、機器毎のIPアドレスとEOJがわかる。

```
Destination IP: 224.0.23.0
SE0J: 0x05FF01
DE0J: 0x0EF001
ESV: 0x62(GET)
OPC: 1
EPC: 0xD6
```

例：ネットワーク内にエアコンと照明がある場合

コントローラ	エアコン
照明	
(IP=192.168.1.1)	(IP=192.168.1.2)
(IP=192.168.1.3)	
(EOJ=0x05FF01)	(EOJ=0x013001)
(EOJ=0x029001)	
IP SRC:192.168.1.1, DST:224.0.23.0	
1081 0001 0EF001 0EF001 62 01 D6 00	IP SRC:192.168.1.1,
DST:224.0.23.0	1081 0001 0EF001
----->	
0EF001 62 01 D6 00	
----->	
IP SRC:192.168.1.2, DST:192.168.1.1	
1081 0001 0EF001 0EF001 72 01 D6 04 01013001	
<-----	

```

|
|                                     | IP SRC:192.168.1.3,
DST:192.168.1.1 |
|                                     | 1081 0001 0EF001
0EF001 72 01 D6 04 01029001 |
| <-----
-----
|

```

2. 機器オブジェクトの規格バージョン情報の取得

機器のプロパティは規格バージョン(Appendix version)によって定義が異なる場合がある。したがって機器の規格バージョン情報プロパティの値をあらかじめ取得しておく必要がある。

```

Destination IP: IP address of the device
SE0J: 0x05FF01
DE0J: E0J of the device object
ESV: 0x62(GET)
OPC: 1
EPC: 0x82

```

例

コントローラ (IP=192.168.1.1) (E0J=0x05FF01)	エアコン (IP=192.168.1.2) (E0J=0x013001)
IP SRC:192.168.1.1, DST:192.168.1.2	
1081 0002 05FF01 013001 62 01 82 00	
----->	
IP SRC:192.168.1.2, DST:192.168.1.1	
1081 0002 013001 05FF01 72 01 82 04 00004B00	
<-----	

3. Node profileの識別番号(EPC=0x83)の取得

機器オブジェクトの識別には、IP addressとE0Jを利用する。機器のIP addressは通常DHCPで設定するため、機器の電源を入れ直した場合や一定時間後に機器のIP addressが変更される場合がある。IP addressと機器の対応を追跡するには、必須プロパティの一つであるNode profileの識別番号(EPC=0x83)を利用する。識別番号は全てのECHONET Lite機器を通じてユニークな値であることが定められている。なお、機器オブジェクトにも識別番号(EPC=0x83)が定義されているが必須ではない。

```

Destination IP: IP address of the device
SE0J: 0x05FF01
DE0J: E0J of the device object
ESV: 0x62(GET)

```

```

OPC:  1
EPC:  0x83

```

例

コントローラ (IP=192.168.1.1) (EOJ=0x05FF01)	エアコン (IP=192.168.1.2) (EOJ=0x013001)
IP SRC:192.168.1.1, DST:192.168.1.2	
1081 0003 05FF01 013001 62 01 83 00	
----->	
IP SRC:192.168.1.2, DST:192.168.1.1	
1081 0003 013001 05FF01 72 01 83 11	
FE000069000102030405060708090A0B0C	
<-----	

4. 機器オブジェクトのプロパティマップ(EPC=0x9D, 9E, 9F)の取得

必須プロパティ以外のプロパティにアクセスする場合、対象の機器がどのプロパティを実装しているかをあらかじめ知っておく必要がある。そのために状態アナウンスプロパティマップ(EPC=0x9D), Setプロパティマップ(0x9E), Getプロパティマップ(0x9F)を取得する。プロパティマップはプロパティ数が15以下の場合にはEPCの値がそのまま列挙されるが、16以上の場合にはbitmapで記述される(reference:APPENDIX 付録1)。

Request:

```

Destination IP: IP address of the device
SE0J: 0x05FF01
DE0J: EOJ of the device object
ESV:  0x62(Get)
OPC:  1
EPC:  0x9D, 0x9E, 0x9F

```

Response(EPCの個数が15以下の場合):

```

ESV:  0x62(Get\_Res)
EDT:  1byte目:EPCの個数、2byte目以降:EPCのリスト
Example of EDT: 04808FB0B3

```

Response(EPCの個数が16以上の場合):

```

ESV:  0x62(Get\_Res)
OPC:  17
EDT:  1byte目:EPCの個数、2byte目以降:EPCを表すbitmap(以下の表を参照)
Example of EDT: 160B0101090000000101010303030303

```

(EPC: 80,81,82,83,87,88,89,8A,8B,8C,8D,8E,8F,90,9A,9B,9C,9D,9E,9F,B0,B3)

bitmap of EPC (bit value 1:implemented, 0:not implemented)

byte\bit	bit-7	bit-6	bit-5	bit-4	bit-3	bit-2	bit-1	bit-0
byte-2	0xF0	0xE0	0xD0	0xC0	0xB0	0xA0	0x90	0x80
byte-3	0xF1	0xE1	0xD1	0xC1	0xB1	0xA1	0x91	0x81
byte-4	0xF2	0xE2	0xD2	0xC2	0xB2	0xA2	0x92	0x82
byte-5	0xF3	0xE3	0xD3	0xC3	0xB3	0xA3	0x93	0x83
byte-6	0xF4	0xE4	0xD4	0xC4	0xB4	0xA4	0x94	0x84
byte-7	0xF5	0xE5	0xD5	0xC5	0xB5	0xA5	0x95	0x85
byte-8	0xF6	0xE6	0xD6	0xC6	0xB6	0xA6	0x96	0x86
byte-9	0xF7	0xE7	0xD7	0xC7	0xB7	0xA7	0x97	0x87
byte-10	0xF8	0xE8	0xD8	0xC8	0xB8	0xA8	0x98	0x88
byte-11	0xF9	0xE9	0xD9	0xC9	0xB9	0xA9	0x99	0x89
byte-12	0xFA	0xEA	0xDA	0xCA	0xBA	0xAA	0x9A	0x8A
byte-13	0xFB	0xEB	0xDB	0xCB	0xBB	0xAB	0x9B	0x8B
byte-14	0xFC	0xEC	0xDC	0xCC	0xBC	0xAC	0x9C	0x8C
byte-15	0xFD	0xED	0xDD	0xCD	0xBD	0xAD	0x9D	0x8D
byte-16	0xFE	0xEE	0xDE	0xCE	0xBE	0xAE	0x9E	0x8E
byte-17	0xFF	0xEF	0xDF	0xCF	0xBF	0xAF	0x9F	0x8F

例

コントローラ
(IP=192.168.1.1)
(EOJ=0x05FF01)

エアコン
(IP=192.168.1.2)
(EOJ=0x013001)

```

| Get Property map(Anno) |
| IP SRC:192.168.1.1, DST:192.168.1.2 |
| 1081 0004 05FF01 013001 62 01 9D 00 |
| -----> |
| IP SRC:192.168.1.2, DST:192.168.1.1 |
| 1081 0004 013001 05FF01 72 01 9D 05 04808FA0B0 |
| <----- |
| Get Property map(Set) |
| IP SRC:192.168.1.1, DST:192.168.1.2 |
| 1081 0004 05FF01 013001 62 01 9E 00 |

```

```

|----->
|
| IP SRC:192.168.1.2, DST:192.168.1.1
| 1081 0004 013001 05FF01 72 01 9E 06 05808FA0B0B3
|<-----
|
| Get Property map(Get)
| IP SRC:192.168.1.1, DST:192.168.1.2
| 1081 0004 05FF01 013001 62 01 9F 00
|----->
|
| IP SRC:192.168.1.2, DST:192.168.1.1
| 1081 0004 013001 05FF01 72 01 9F 07 06808FA0B0B3BB
|<-----

```

5. メーカーコード(EPC=0x83)の取得

機器オブジェクトのメーカー名が分かると、機器の特定に利用できる場合がある。必須プロパティの一つであるメーカーコード(EPC=0x8A)を取得することでメーカー名がわかる。

```

Destination IP: IP address of the device
SE0J: 0x05FF01
DE0J: E0J of the device object
ESV: 0x62(GET)
OPC: 1
EPC: 0x8A

```

例

コントローラ (IP=192.168.1.1) (E0J=0x05FF01)	エアコン (IP=192.168.1.2) (E0J=0x013001)
IP SRC:192.168.1.1, DST:192.168.1.2	
1081 0004 05FF01 013001 62 01 8A 00	
----->	
IP SRC:192.168.1.2, DST:192.168.1.1	
1081 0004 013001 05FF01 72 01 8A 03 000068	
<-----	

メーカーコードは3 byteであり、メーカーコードとメーカー名の対応表はエコーネットコンソーシアムの会員のみ公開されている。実機からメーカーコードを取得すればメーカー名との対応は明らかなので、神奈川工科大学が所有する機器のメーカーコードとメーカー名の対応表を第6章に示す。

2.8 異常処理

ECHONET Liteのリクエストメッセージ(ESV=0x60, 61, 62, 63, 6E)を受信した時の異常処理を説明する(reference:第2部4.2.2, 第2部付録1, システム設計指針2.2)。条件に応じて以下のどれかの処理を行う。

- レスポンスを送信しない。命令を無視する。
- メッセージの受理処理(*)を行う。命令を無視する。
- SNAを送信する。命令を無視する。

(*)ESV=0x60の場合は「レスポンスを送信しない。」、ESV=0x61の場合は「レスポンスESV=0x71を送信する」、ESV=0x6Eの場合は「レスポンスESV=0x7Eを送信する」。

2.8.1 Message frame error

条件：メッセージがECHONET Liteのフレームフォーマットに適合しない場合。

処理：レスポンスを送信しない。命令を無視する。

備考：SetGetはフレーム構造が異なることに注意すること。

2.8.2 EHD error

条件：EHDが0x1081でない場合。

処理：レスポンスを送信しない。命令を無視する。

2.8.3 DEOJ error

条件：DEOJで指定されたオブジェクトが存在しない場合。

処理：レスポンスを送信しない。命令を無視する。

備考：インスタンスコードが0x00の場合の処理に注意。

(reference:第2部付録1)

2.8.4 ESV error

条件：ESVが規格で定義された値以外の場合。

処理：レスポンスを送信しない。命令を無視する。

(reference:第2部付録1)

2.8.5 EPC error

条件：EPCが、指定されたESVの処理を実装しない場合。

処理：SNAをレスポンスする。命令を無視する。

(reference:第2部付録1)

2.8.6 EDT error

条件：SetでEDTのサイズが規格と異なる場合

処理：以下のどちらの処理でも構わない。

(A)：レスポンスを送信しない。命令を無視する。

(B)：SNAをレスポンスする。命令を無視する。

(reference:第2部付録1)

条件：SetでEDTの値が規格外の場合（例：仕様の値域外の値、状態として定義されていない値）

処理（推奨）：メッセージ受理の処理(*)を行う。命令を無視する。

処理（これでも構わない）：SNAをレスポンスする。命令を無視する。

(*)ESV=0x60の場合は「レスポンスを送信しない。」、ESV=0x61の場合は「レスポンスESV=0x71を送信する」。

る」、ESV=0x6Eの場合は「レスポンスESV=0x7Eを送信する」。
(reference:システム設計指針2.2)

Get_SNA をレスポンスする例 (PDC=0x00でレスポンスする)

```

リクエスト：コントローラ(0x05FF01) -> エアコン(0x013001)
10 81 00 01 05 FF 01 01 30 01 62 01 A2 00
<----> <----> <-----> <-----> <-> <-> <-> <->
EHD   TID   SE0J   DE0J   ESV OPC EPC PDC

レスポンス：エアコン(0x013001) -> コントローラ(0x05FF01)
10 81 00 01 01 30 01 05 FF 01 52 01 A2 00
<----> <----> <-----> <-----> <-> <-> <-> <->
EHD   TID   SE0J   DE0J   ESV OPC EPC PDC

```

SetC_SNA をレスポンスする例 (SetCのEDT値をレスポンスする)

```

リクエスト：コントローラ(0x05FF01) -> エアコン(0x013001)
10 81 00 01 05 FF 01 01 30 01 61 01 80 02 30 00
<----> <----> <-----> <-----> <-> <-> <-> <-> <---->
EHD   TID   SE0J   DE0J   ESV OPC EPC PDC EDT

レスポンス：エアコン(0x013001) -> コントローラ(0x05FF01)
10 81 00 01 01 30 01 05 FF 01 51 01 80 02 30 00
<----> <----> <-----> <-----> <-> <-> <-> <-> <---->
EHD   TID   SE0J   DE0J   ESV OPC EPC PDC EDT

```

2.9 プロパティの値と実装

EDTが状態の場合

機器は規格で定義されている全ての状態を実装する必要はない。

条件：実装していない状態のSetリクエストを受信した場合

処理（推奨）：メッセージの受理処理(*)を行う。命令を無視する。(reference:システム設計指針2.1(3), 2.2)

処理（これでも構わない）：SNAをレスポンスする。命令を無視する。(reference:システム設計指針2.2)

(*)ESV=0x60の場合は「レスポンスを送信しない。」、ESV=0x61の場合は「レスポンスESV=0x71を送信する」、ESV=0x6Eの場合は「レスポンスESV=0x7Eを送信する」。

例：エアコンのEPC=0xB0(運転モード)の状態は 0x41(自動),0x42(冷房),0x43(暖房),0x44(除湿),0x45(送風),0x40(その他)が定義されている。冷房専用のエアコンがメッセージ(ESV=SetC, EPC=0xB0, EDT=0x43)を受信した場合、Set_Resをレスポンスするが、内部の動作モードの変更は行わない。プロパティの値も変更しない。

EDTがレベルの場合

機器は規格で定義されている全てのレベル値を内部状態として実装する必要はない。

例えばエアコンのEPC=0xA0(風量設定)など、強弱設定を8段階のレベル(0x31~0x38)で指定するプロパティが多く存在する。このようなプロパティにおいて、内部状態を弱・中・強の3段階しか持たない場合、EDTの全てのレベル値を実機のいずれかの状態にマッピングすることが推奨される(reference:システム設計指針2.2)。以下にEDTの値と内部状態のマッピング例を示す。

EDT値	0x31	0x32	0x33	0x34	0x35	0x36	0x37	0x38
内部状態	弱	弱	中	中	中	中	強	強

Getの動作に関しては以下の2通りの実装が考えられる。コントローラの立場としては(A)の実装が望ましい。

(A) Setした値がレスポンスされる。

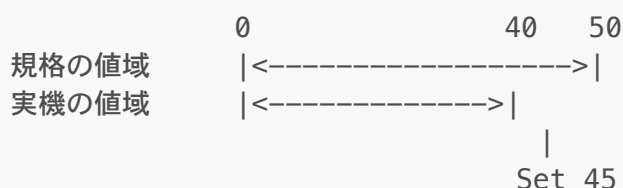
(B) 内部状態に対応した代表値（例：0x31(弱), 0x35(中), 0x38(強))がレスポンスされる(reference:システム設計指針2.1(2))

上記のマッピング例でSet(EDT=0x33)を実行すると、内部状態は「中」になる。ここでGetを実行すると(A)の場合は0x33が、(B)の場合は0x35がレスポンスされる。

EDTが数値の場合

A. 実機器の値域が規格の値域の内側の場合

Set要求のEDT値が規格内の値であるが実装の値を超える場合、実装の端の値を設定する(reference:システム設計指針2.1(1))。例えば下図のように規格の値域の最大値が50かつ実装の値域の最大値が40の場合にSetC(EDT=45)を受信したときには、プロパティの値を40とする(reference:システム設計指針2.1(1))。ただし、実装の都合でSNAをレスポンスし、命令を無視しても構わない(reference:システム設計指針2.2)。



B. 実機器の値域が規格の値域を超える場合

内部状態が規格の値域を超える時にGet要求を受信した場合、OverflowまたはUnderflowに対応する値をレスポンスする(reference:第2部6.2.2(2))。例えば下図のように規格の値域の最大値が50かつ実装の値域の最大値が60で内部状態が55の場合にGetを受信したときには、Overflowに対応する値0xFFをレスポンスする。



2.10 複数命令の処理

EPC, PDC, EDTのセットを命令(operation)と呼ぶ。1つのECHONET Liteメッセージに複数のoperationを記述することができる。例えばESV=0x62(Get)でEPC=0x80, 0x81, 0x82の3つのプロパティの読み出しリクエストを一度に送信することができる。一度に送るoperationの個数をOPCに設定する(reference:第2部 4.2.3)。エラー処理が複雑なのでプロトタイピング時にはOPC=0x01だけの実装も可能であるが、AIF認証では複数OPCのメッセージの送受信に関する試験がある。

Get: 正常処理

リクエストは、EPCとPDC(=0x00)の組み合わせをOPCが示す回数繰り返して記述する。

レスポンスはEPC, PDC, EDTの組み合わせをOPCが示す回数繰り返して記述する。

以下はOPC=0x03の場合の例を示す。

リクエスト：コントローラ(0x05FF01) → エアコン(0x013001)

```
10 81 00 01 05 FF 01 01 30 01 62 03 80 00 81 00 82 00
<----> <----> <-----> <-----> <--> <--> <---> <---> <---> <---> <---> <--->
EHD   TID   SE0J   DE0J   ESV OPC EPC1 PDC1 EPC2 PDC2 EPC3 PDC3
```

レスポンス：エアコン(0x013001) → コントローラ(0x05FF01)

```
10 81 00 01 01 30 01 05 FF 01 72 03 80 01 31 81 01 00 82 04 00004400
<----> <-----> <-----> <--> <--> <---> <---> <---> <---> <---> <---> <---> <--->
-> <----->
EHD   TID   SE0J   DE0J   ESV OPC EPC1 PDC1 EDT1 EPC2 PDC2 EDT2 EPC3
PDC3 EDT3
```

Get: 異常処理

一つでもoperationが処理できない場合はESV=0x52(Get_SNA)でレスポンスする。

処理できないEPCはPDC=0x00としてレスポンスする。

以下は3番目のoperationが異常処理の例を示す。

リクエスト：コントローラ(0x05FF01) → エアコン(0x013001)

```
10 81 00 01 05 FF 01 01 30 01 62 03 80 00 81 00 A2 00
<----> <----> <-----> <-----> <--> <--> <---> <---> <---> <---> <---> <--->
EHD   TID   SE0J   DE0J   ESV OPC EPC1 PDC1 EPC2 PDC2 EPC3 PDC3
```

レスポンス：エアコン(0x013001) → コントローラ(0x05FF01)

```
10 81 00 01 01 30 01 05 FF 01 52 03 80 01 31 81 01 00 A2 00
<----> <----> <-----> <-----> <--> <--> <---> <---> <---> <---> <---> <---> <---> <--->
->
EHD   TID   SE0J   DE0J   ESV OPC EPC1 PDC1 EDT1 EPC2 PDC2 EDT2 EPC3
PDC3
```

Set: 正常処理

SetIの場合、全てのoperationを受け付けた場合はレスポンスは不要。

以下はSetCの場合の例を示す。

```

リクエスト：コントローラ(0x05FF01) -> エアコン(0x013001)
10 81 00 01 05 FF 01 01 30 01 61 02 80 01 30 81 01 08
<---> <---> <-----> <-----> <-> <-> <---> <---> <---> <---> <---> <--->
EHD   TID   SE0J   DE0J   ESV OPC EPC1 PDC1 EDT1 EPC2 PDC2 EDT2

レスポンス：エアコン(0x013001) -> コントローラ(0x05FF01)
10 81 00 01 01 30 01 05 FF 01 71 02 80 00 81 00
<---> <---> <-----> <-----> <-> <-> <---> <---> <---> <--->
EHD   TID   SE0J   DE0J   ESV OPC EPC1 PDC1 EPC2 PDC2

```

Set: 異常処理

一つでもoperationが処理できない場合はESV=0x50(SetI_SNA)または0x51(SetC_SNA)でレスポンスする。

処理できたEPCはPDC=0x00、処理できないEPCはリクエストされたEDT値をレスポンスする。

以下は1番目のoperationが異常処理の例を示す。

```

リクエスト：コントローラ(0x05FF01) -> エアコン(0x013001)
10 81 00 01 05 FF 01 01 30 01 61 02 80 01 32 81 01 08
<---> <---> <-----> <-----> <-> <-> <---> <---> <---> <---> <---> <--->
EHD   TID   SE0J   DE0J   ESV OPC EPC1 PDC1 EDT1 EPC2 PDC2 EDT2

レスポンス：エアコン(0x013001) -> コントローラ(0x05FF01)
10 81 00 01 01 30 01 05 FF 01 51 02 80 01 32 81 00
<---> <---> <-----> <-----> <-> <-> <---> <---> <---> <---> <--->
EHD   TID   SE0J   DE0J   ESV OPC EPC1 PDC1 EDT1 EPC2 PDC2

```

2.11 スマート電力量メータ

低圧スマート電力量メータ(E0J=0x0288)と高圧スマートメータ(E0J=0x028A)に関する注意事項を述べる。

低圧スマート電力量メータ(E0J=0x0288)

- 物理レイヤーはWi-SUN(無線)またはG3-PLC(有線)
- IPv6で一对一の接続のみ
- PANA 認証

高圧スマートメータ(E0J=0x028A)

- 物理レイヤーは有線LAN
- IPv6

3. 発展編

3.1 SetGet

ESV=0x6E(SetGet)を使うとSetとGetのoperationを一つのメッセージで送信することができる(reference:第2部4.2.3.4)。ただしメッセージのフレーム構成が他のものと異なる。異常処理はかなり複雑である。SetGetの処理の実装はオプションであり、実装しない場合はSetGet_SNA(OPCSet=0, OPCGet=0)をレスポンスする。

SetGetを使用したメッセージの中でSetとGetで同一のEPCを指定した場合、SetとGetのどちらが先に実行されるかは実装依存である(reference:第2部4.2.3.4)。一方、システム設計指針ではSetの処理後にGetの処理を行うことを実装の指針としている (reference:システム設計指針2.6)。

メッセージのフレーム構成

OPC_Setの後にSetCのoperation(EPC, PDC, EDT)が列挙され、その後OPC_Getの後にGetのoperation(EPC, PDC)が列挙される。

リクエスト

```

+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
-+---+---+---+---+---+
|EHD|TID|SE0J|DE0J|ESV
|OPCSet|EPC1|PDC1|EDT1|...|EPCm|PDCm|EDTm|OPCGet|EPC1|PDC1|...|EPCn|PDCn| | | | |
|  |  |  |  |  |0x6E| (m) |  |  |  |  |  |  |  |  |  |  | (n) |
|0x00|  |  |0x00|
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
-+---+---+---+---+---+

```

レスポンス

```

+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
-+---+---+---+---+---+
|EHD|TID|SE0J|DE0J|ESV
|OPCSet|EPC1|PDC1|...|EPCm|PDCm|OPCGet|EPC1|PDC1|EDT1|...|EPCn|PDCn|EDTn| |
|  |  |  |  |  |0x7E| (m) |  |0x00|  |  |0x00| (n) |  |  |
|  |  |  |  |  |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
-+---+---+---+---+---+

```

正常処理の例：OPC_Set=2, OPC_Get=2

レスポンスは、Setの場合はPDC=0x00、Getの場合はプロパティの値

```

リクエスト：コントローラ(0x05FF01) -> エアコン(0x013001)
10 81 00 01 05 FF 01 01 30 01 6E 02      80 01 30 81 01 08 02
A0 00 B0 00
<---> <---> <-----> <-----> <-> <-----> <---> <---> <---> <---> <---> <---> <--->
-> <---> <---> <---> <--->
EHD  TID  SE0J  DE0J  ESV OPCSet EPC1 PDC1 EDT1 EPC2 PDC2 EDT2
OPCGet EPC1 PDC1 EPC2 PDC2

レスポンス：エアコン(0x013001) -> コントローラ(0x05FF01)

```

```

10 81 00 01 01 30 01 05 FF 01 7E 02      80 00 81 00 02      A0 01
41 B0 01 42
<----> <----> <-----> <-----> <--> <-----> <--> <--> <--> <--> <-----> <--> <--
-> <--> <--> <--> <-->
EHD TID SE0J DE0J ESV OPCSet EPC1 PDC1 EPC2 PDC2 OPCGet EPC1
PDC1 EDT1 EPC2 PDC2 EDT2

```

異常処理の例：OPC_Set=2, OPC_Get=2

一つでもoperationが処理できない場合はESV=0x5E(SetGet_SNA)とし、Setが処理できない場合はSetのEDT値を、Getが処理できない場合はPDC=0x00としてレスポンスする。以下はSetの1番目とGetの2番目のoperationが異常処理の例である。

```

リクエスト：コントローラ(0x05FF01) -> エアコン(0x013001)
10 81 00 01 01 30 01 05 FF 01 01 30 01 6E 02      80 01 30 81 01 08 02
A0 00 B0 00
<----> <----> <-----> <-----> <--> <-----> <--> <--> <--> <--> <-----> <--> <--
-> <--> <--> <--> <-->
EHD TID SE0J DE0J ESV OPCSet EPC1 PDC1 EDT1 EPC2 PDC2 EDT2
OPCGet EPC1 PDC1 EPC2 PDC2

レスポンス：エアコン(0x013001) -> コントローラ(0x05FF01)
10 81 00 01 01 30 01 05 FF 01 5E 02      80 01 30 81 00 02      A0
01 41 B0 00
<----> <----> <-----> <-----> <--> <-----> <--> <--> <--> <--> <-----> <--
-> <--> <--> <--> <-->
EHD TID SE0J DE0J ESV OPCSet EPC1 PDC1 EDT1 EPC2 PDC2 OPCGet
EPC1 PDC1 EDT1 EPC2 PDC2

```

SetGet の処理を実装しない場合

SetGetの処理の実装はオプションである。処理を実装しない場合、SetGetに対してSetGet_SNA(OPCSet=0, OPCGet=0)のレスポンスをする。

```

リクエスト：コントローラ(0x05FF01) -> エアコン(0x013001)
10 81 00 01 01 30 01 05 FF 01 01 30 01 6E 02      80 01 30 81 01 08 02
A0 00 B0 00
<----> <----> <-----> <-----> <--> <-----> <--> <--> <--> <--> <-----> <--> <--
-> <--> <--> <--> <-->
EHD TID SE0J DE0J ESV OPCSet EPC1 PDC1 EDT1 EPC2 PDC2 EDT2
OPCGet EPC1 PDC1 EPC2 PDC2

レスポンス：エアコン(0x013001) -> コントローラ(0x05FF01)
10 81 00 01 01 30 01 05 FF 01 5E 00      00
<----> <----> <-----> <-----> <--> <-----> <----->

```

EHD	TID	SE0J	DE0J	ESV	OPCSet	OPCGet
-----	-----	------	------	-----	--------	--------

3.2 現実的な実装について

実験用のプロトタイピングと製品化前提のケースで、ECHONET Liteの現実的な実装について説明する。プロトタイピングでは動作確認のための最低限の実装とする。製品化前提の場合には認証を考慮する必要がある。認証に関しては[第4章](#)を参照のこと。

3.2.1 コントローラ（プロトタイピング）

コントローラはGetやSetCの命令を受信してレスポンスする必要は基本的に無いので、とりあえず実装しなくても構わない。ただし、制御対象の機器がコントローラの素性を確認する場合には、以下のEPCのGet命令を受信した場合にレスポンスすることをお勧めする。

Node profile

EPC	プロパティ名	EDT値 (例)
0x82	Version情報	0x010D0100: V1.13
0x83	識別番号	0xFEFFFFFF000102030405060708090A0B0C
0x8A	メーカーコード	0xFFFFF: 実験用のコード (reference:Appendix 2.6)

Super class

EPC	プロパティ名	EDT値 (例)
0x80	動作状態	0x30: ON
0x8A	メーカーコード	0xFFFFF: 実験用のコード (reference:Appendix 2.6)

コントローラが送信するコマンドはGetとSetCを利用し、OPC=1とする。

3.2.2 機器（プロトタイピング）

コントローラが製品である場合、機器はNode profileと機器オブジェクトの必須プロパティの実装が必要。

Node profileのEPC=0x83（識別番号）は0xFE(1byte) + メーカーコード(3byte) + Unique ID(13byte)で構成される。以下の値は、実験用のメーカーコード0xFFFFFとUnique ID:000102030405060708090A0B0C の例である。ネットワーク内でこの機器を1台しか使わない場合はこの値でよいが、複数台使う可能性がある場合はそれぞれの機器の識別番号をユニークな値(*)にすること。同一の値の場合、コントローラが誤動作する可能性がある。

(*) 値をマニュアルで変更するか、乱数やMACアドレスを利用してユニークな値をプログラムで生成する。

プロパティの値の例

Node profile

EPC	プロパティ名	EDT値 (例)
0x80	動作状態	0x30:ON
0x82	Version情報	0x010D0100: V1.13
0x83	識別番号	0xFEFFFFFF000102030405060708090A0B0C
0x8A	メーカコード	0xFFFFFFFF: 実験用のコード (reference:Appendix 2.6)
0x9D	状態アナウンスプロパティマップ	0x0280D5
0x9E	Set プロパティマップ	0x00
0x9F	Get プロパティマップ	0x0B8082838A9D9E9FD3D5D6D7
0xD3	自ノードインスタンス数	0x000001
0xD4	自ノードクラス数	0x0002
0xD5	インスタンスリスト通知	0x01013001
0xD6	自ノードインスタンスリストS	0x01013001
0xD7	自ノードクラスリストS	0x010130

機器オブジェクトのSuper class

EPC	プロパティ名	EDT値 (例)
0x80	動作状態	0x30: ON
0x81	設置場所	0x00: 未設定
0x82	規格Version情報	0x00004C00: Release L
0x88	異常発生状態	0x42: 異常なし
0x8A	メーカコード	0xFFFFFFFF: 実験用のコード (reference:Appendix 2.6)
0x9D	状態アナウンスプロパティマップ	実装に合わせる
0x9E	Setプロパティマップ	実装に合わせる
0x9F	Getプロパティマップ	実装に合わせる

実装は、機器が受信するサービスのうちGetとSetCに対応し、Get_ResとSet_Resをレスポンスする。OPCは1のみ対応する。エラー処理は特に必要ない。起動時処理のNode profileのEPC=0xD5のINFは実装する。

3.2.3 コントローラ (製品化前提)

- Node profileオブジェクトと機器オブジェクトの必須プロパティは実装が必要。
- Node profileオブジェクトのEPC=0xBF (個体識別情報) は実装しない(*)。
- 機器オブジェクトのEPC=0x81(設置場所)のプロパティ値は1byteと17byteで定義されているが、1byteの実装が主流。
- 機器オブジェクトのEPC=0x83(識別番号) は必須ではないが実装する。

- リクエスト送信に使用するESVは 0x61(SetC), 0x62(Get)。SetI, INF_REQ, SetGetは使わない。
- 通知送信に使用するESVは 0x73(INF)。0x74(INFC)は使わない。
- Getを送信後レスポンスを受信するまでに時間がかかるプロパティが存在する。例えば低圧スマート電力量メータの履歴値など。タイムアウトはプロパティごとに考慮が必要。
- SetCを送信後、Set動作を確認するためにプロパティ値をGetするのが望ましい。Setした値とGetした値が異なる場合があるので注意。
 - 値が変わらない場合
 - Set動作の完了に時間がかかる場合
 - Setした値を機器が実装していない場合
 - 値が変わっているが、Setした値でない場合
 - 機器側の実装の都合
- リクエストの受信は全てのESV(0x60,0x61,0x62,0x63,0x6E)に対応する必要がある。
- INFC(0x74)を受信したら、EPC, EDTの値にかかわらずINFC_Res(0x7A)をレスポンスする。
- SetGetを受信した場合、SetGet_SNA(OPCSet=0, OPCGet=0)をレスポンスする。
- 送信コマンドの複数OPC対応が必要かどうかは、対応するAIF認証仕様を確認すること。

(*) ECHONET Lite V1.01までは必須であった。Node profileを識別するために規定されたが、仕様が複雑すぎるので、V1.1以降はEPC=0x83がNode profileを識別するために使われることになった。

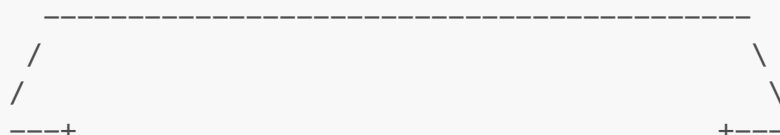
3.2.4 機器（製品化前提）

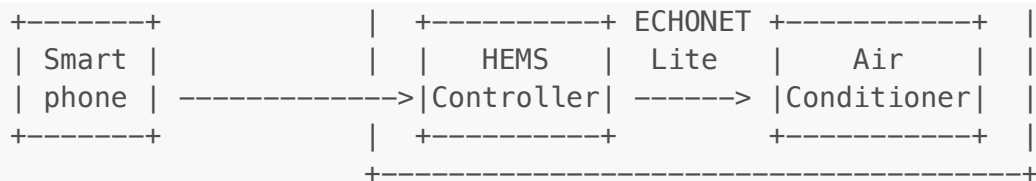
- Node profileオブジェクトと機器オブジェクトの必須プロパティは実装が必要
- Node profileオブジェクトのEPC=0xBF:個体識別情報は実装しない(*)
- 機器オブジェクトのEPC=0x81(設置場所)のプロパティ値は1byteと17byteで定義されているが、1byteの実装が主流。
- 機器オブジェクトのEPC=0x83(識別番号) は必須ではないが実装を推奨する(reference:システム設計指針2.8)。
- 通知送信に使用するESVは 0x73(INF)。0x74(INFC)は使わない。
- リクエストの受信は全てのESV(0x60,0x61,0x62,0x63,0x6E)に対応する必要がある。
- INFC(0x74)を受信したら、EPC, EDTの値にかかわらずINFC_Res(0x7A)をレスポンスする。
- SetGetを受信した場合、SetGet_SNA(OPCSet=0, OPCGet=0)をレスポンスする。
- 受信コマンドの複数OPC対応が必要かどうかは対応するAIF認証仕様を確認すること。

(*) ECHONET Lite V1.01までは必須であった。Node profileを識別するために規定されたが、仕様が複雑すぎるので、V1.1以降はEPC=0x83がNode profileを識別するために使われることになった。

3.3 遠隔操作設定

機器オブジェクトのSuper classのEPC=0x93（遠隔操作設定）プロパティは、電気用品安全法の遠隔操作に関連する項目である(reference:APPENDIX2.16, システム設計指針第5章)。例えば下図のように宅外からスマートフォンを利用して宅内のエアコンを制御する場合に、HEMS Controllerとエアコンの間の通信にECHONET Liteを利用するシステム構成を想定している。遠隔操作設定プロパティは必須プロパティではなく、実装もめんどろななのでおすすめではないが、以下に遠隔操作設定の説明を行う。





コントローラ側の実装

コントローラがインターネットなど宅外からの制御を受信し、宅内の機器をエコーネットライトで制御(Set)する場合は、EPC=0x93, EDT=0x42（公衆回線経由操作）の命令を1st operationとして追加する。以下にエアコンの電源をON遠隔操作でONにする場合の例を示す。

```
1081 0001 05FF01 013001 61 02 930142 800130
```

- 宅外からの命令でECHONET LiteのGetの操作をする場合は、遠隔操作プロパティを追加しない。
- コントローラのGUIでの操作などの宅内制御の場合は、遠隔操作プロパティを追加しない。

機器側の実装

- 受信したSetコマンドに、EPC=0x93, EDT=0x42（公衆回線経由操作）命令がある場合
 - 遠隔操作プロパティ(EPC=0x93)の値を、0x42:公衆回線経由操作にする。
 - 機器ごとに操作内容を判断して動作する(法律の解釈も関係するので実装は自己責任)。
- 受信したSetコマンドにEPC=0x93命令がない場合
 - 遠隔操作プロパティ(EPC=0x93)の値を、0x41:公衆回線未経由操作にする。
 - Setの動作を行う。
- 機器のパネルやリモコンでの操作を行った場合
 - 遠隔操作プロパティ(EPC=0x93)の値を、0x41:公衆回線未経由操作にする。

3.4 送信専用ノード

電池駆動のセンサーなどを想定して送信専用機器の扱いが定義されている。

- 送信専用ノードのProfile node objectのEOJは0x0EF002(reference:第2部4.3.1)
- 定期的にProfile node objectのインスタンスリスト(EPC=0xD6)をINFでMulticastする(reference:システム設計指針:2.10)
- 機器オブジェクト詳細規定の必須の項目の対応は必要ではない（reference:第2部6.2.3)

3.5 ECHONET Liteのバージョン間の違い

2019年8月時点でこれまでECHONET Liteは以下のバージョンがリリースされている。バージョン間の違いに関して留意すべき項目を記述する。

```
Version 1.00, 1.01, 1.10, 1.11, 1.12, 1.13
```

Version	留意すべき項目
1.00 -> 1.01	特になし
1.01 -> 1.10	- ノードプロファイル EPC=0xBF(個体識別情報)のアクセスルールに関して、V1.01までは必須。V1.10は必須ではない(reference:第2部6.11.1表6-7)。 - ノードプロファイル EPC=0x83(識別番号)のデータフォーマットに関して、V1.01までは 9 or 17byte。V1.10は17byteのみ。 - コントローラによる機器探索のシーケンスに関して、Version 1.01までよく行なわれていた方式： ノードプロファイル EPC=0xD5(インスタンスリスト)にINF_REQをマルチキャスト Ver. 1.10で明記された推奨方式(reference:第5部)： ノードプロファイル EPC=0xD6(インスタンスリスト S)にGETをマルチキャスト - 送信専用機器のDEOJの値に関して、V1.01までは特別な記述なし。V1.10は0x0EF002(reference:第2部6.11.1)。
1.10 -> 1.11	- WAN経由の操作に関して指針の追加(reference:第5部6章)。
1.11 -> 1.12	特になし
1.12 -> 1.13	- 第5部の中身をすべて「ECHONET Liteシステム設計指針」として、ECHONET Lite規格から独立させた。

4. 認証制度

4.1 エコーネットコンソーシアム

[エコーネットコンソーシアム](#)はECHONET Liteの規格を策定する団体である。国際標準化や認証制度策定などの活動も行っている。ECHONET Liteの規格は公開され無償で利用できるが、認証を受けるためにはコンソーシアムの会員である必要がある。

[認証制度](#)には「ECHONET Lite 規格認証制度」と「AIF仕様認証制度」がある。認証に関する仕様書はエコーネットコンソーシアムの会員のみに公開されている。

4.2 ECHONET Lite 規格認証制度

ECHONET Lite 規格認証はECHONET Liteの規格の中で全ての機器に共通の項目が正しく実装されているかを確認する。コンソーシアムは試験ツールを提供していない。試験項目の例を以下に示す。

- Node profileのプロパティのアクセス
- 機器オブジェクトのスーパークラスのプロパティのアクセス
- 起動時のインスタンスリスト通知の確認

4.3 AIF仕様認証制度

AIF仕様認証は、機器オブジェクトごとのプロパティにアクセスしその値を確認する。コンソーシアムは会員に対して試験ツールを提供している。2019年8月時点で以下のカテゴリーの機器とそれを制御するコントローラは、ECHONET Lite 規格認証を取得したのちにAIF仕様認証を取得することが求められる。

- 低圧/高圧スマート電力量メータ
- 家庭用エアコン

- 住宅用太陽光発電
- 瞬間式給湯器
- 照明機器
- 蓄電池
- 電気自動車充放電器／電気自動車充電器
- 電気自動車充放電器
- 燃料電池
- 業務用パッケージエアコン
- 業務用ショーケース
- 照明システム
- 拡張照明システム

機種によっては、AIF仕様で搭載必須のプロパティが追加されている場合もあるので注意すること。また複数 OPCを前提の試験項目もある。

付録. メーカーコード

神奈川工科大学が所有する機器のメーカーコードとメーカー名の対応表

Manufacture code	Manufacture name
0x000005	シャープ株式会社
0x000006	三菱電機株式会社
0x000008	ダイキン工業株式会社
0x000009	日本電気株式会社
0x00000B	パナソニック株式会社
0x000016	株式会社東芝
0x00001B	東芝ライテック株式会社
0x000022	日立アプライアンス株式会社
0x00004E	富士通株式会社
0x000053	株式会社ユビキタス
0x000059	リンナイ株式会社
0x00005E	株式会社GWソーラー
0x000060	株式会社ソニーコンピュータサイエンス研究所
0x000063	河村電器産業株式会社
0x000064	オムロン株式会社
0x000069	東芝ライフスタイル株式会社
0x00006C	ニチコン株式会社
0x00006F	株式会社バッファロー

Manufacture code	Manufacture name
0x000072	株式会社エナリス
0x000077	神奈川工科大学
0x000078	日立マクセル株式会社
0x00007D	POWERTECH INDUSTRIAL
0x000086	西日本電信電話株式会社
0x00008A	株式会社富士通ゼネラル
0x0000A3	中部電力株式会社
0x0000AA	株式会社テンフィートライト
0x0000AE	四国電力株式会社
0x0000B5	中国電力株式会社
0x0000B6	文化シヤッター株式会社
0x0000B8	北海道電力株式会社